COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	07/28/2026	**DATE SUBMITTED:**	08/03/2026
NAME:	Troy Jerron A. Bautista	**INSTRUCTOR:**	Sir Jimlord Quejado

ACTIVITY NO. 4.1 **Stacks**

1. PROCEDURE OUTPUT

Program:

```cpp
#include <iostream>
#include <stack> // call the stack from the STL

int main() {
    std::stack<int> newStack; // create an object
    std::stack<int> newerStack;
    std::cout <<"Is the stack empty? " << newStack.empty() << std::endl; // check if the stack is empty

    // insert 3 elements
    newStack.push(3);
    std::cout <<"Top of stack: " << newStack.top() << std::endl;
    newStack.push(2);
    std::cout <<"Top of stack: " << newStack.top() << std::endl;
    newStack.push(1);
    std::cout <<"Top of stack: " << newStack.top() << std::endl;

    std::cout <<"Is the stack empty? " << newStack.empty() << std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);
    std::cout <<"Size of the stack: " << newStack.size() << std::endl;

    // swap
    std::cout <<"\nSwap: " <<std::endl;
    newStack.swap(newerStack);
    std::cout <<"Size of the stack after swap: " << newStack.size() << std::endl;
    newerStack.swap(newStack);
    std::cout <<"Size of the stack after reverting swap: " << newStack.size() << std::endl;

    // emplace
    std::cout <<"\nEmplace: " <<std::endl;
    newStack.emplace(9);
    std::cout <<"Top of stack: " << newStack.top() << std::endl;
    std::cout <<"Size of the stack: " << newStack.size() << std::endl;

    return 0;
}
```

Output:

```
Is the stack empty? 1
Top of stack: 3
Top of stack: 2
Top of stack: 1
Is the stack empty? 0
Size of the stack: 6

Swap:
Size of the stack after swap: 0
Size of the stack after reverting swap: 6

Emplace:
Top of stack: 9
Size of the stack: 7
```

Analysis:

This code demonstrates the stack as methods in C++. Using push(), pop(), isEmpty(), isFull(), swap(), and emplace(), we know how each of the methods work and apply it to our codes in the future.

Table 1: Output of ILO A

Program

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;
int choice;

//prototyping
bool isEmpty();
bool isFull();
void push();
void pop();
void Top();
// displayAll();
void displayAll();

int main() {
    // asked the user for the maximum elements to be inserted
    std::cout << "Enter the number of elements of the new stack (max: 100): ";
    std::cin >> elements;

    while(true) {
        std::cout <<"Stack Operations: " <<std::endl;
        std::cout <<"1. PUSH \n2. POP \n3. Top \n4. isEmpty \n5. isFull \n6. displayAll" <<std::endl;
        std::cin >> choice;

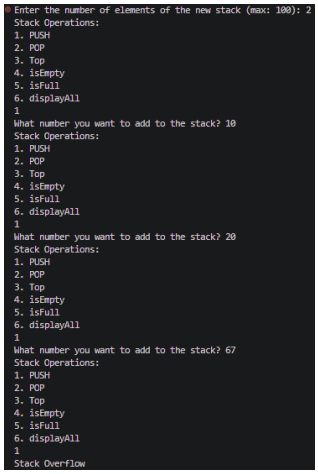
        switch(choice) {
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout <<isEmpty() <<std::endl;
            break;
            case 5: std::cout <<isFull() <<std::endl;
            break;
            case 6: displayAll();
            break;
            default: std::cout <<"Invalid choice!" <<std::endl;
            break;
        }
    }
    return 0;
}

bool isEmpty() {
    // top is -1
    if (top == -1) {
        return true;
    }
    return false;
}

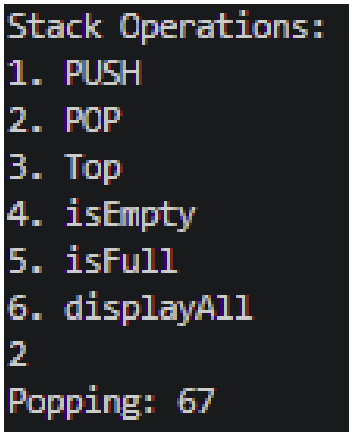
bool isFull() {
    // not equal to the maxCap
    if (top >= elements) {
        std::cout <<"Stack Overflow" <<std::endl;
        return true;
```

Output:

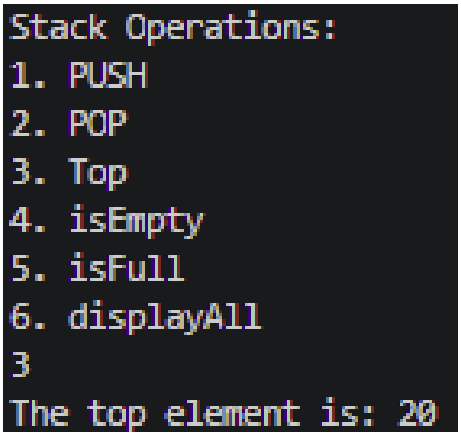
Push:



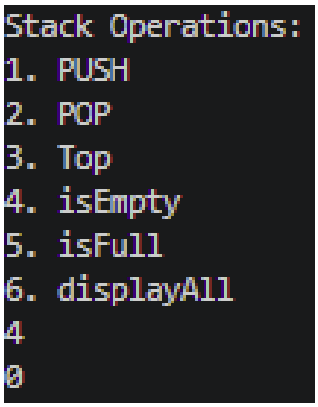
Pop:



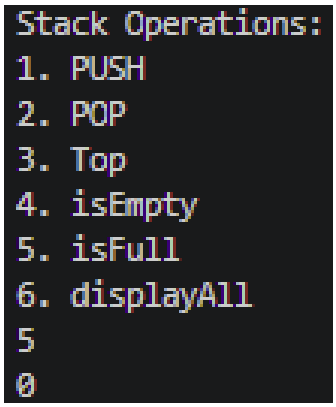
Top:



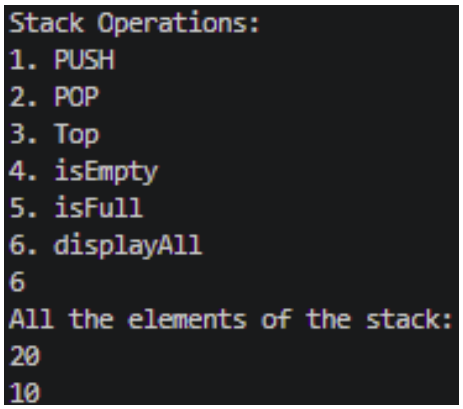
isEmpty:



isFull:



displayAll:



Analysis:

This program lets you create an array and do the following methods, and display all the elements. It also has functions that will determine of your stack is empty or full has an invalid choice when inputting a choice not in the main menu.

Table 1: Output of ILO B

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

a. Arrays

```cpp
#include <iostream>

// set a constant limit for the stack
char ParenStack[100];
int top = -1;

// prototyping
bool isEmpty();
bool BalancedSymbols(char open, char close);
void push(char c);
char pop();

// main/driver code
int main() {
    // input an expression
    char s[100];
    std::cout << "Enter expression: ";
    std::cin >> s;

    // determining if expression is balanced
    bool balanced = true;
    for (int i = 0; s[i] != '\0'; i++) {
        char ch = s[i];
        if (ch == '(' || ch == '{' || ch == '[') {
            push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            char open = pop();
            if (!BalancedSymbols(open, ch)) {
                balanced = false;
                break;
            }
        }
    }
    // output if expression is balanced or not
    if (balanced && top == -1)
        std::cout << "The expression is balanced.";
    else
        std::cout << "The expression is not balanced.";
    return 0;
}

// function if top is -1
bool isEmpty() {
    // top is -1
    if (top == -1) {
        return true;
    }
    return false;
}

// function to determine if expression is balanced
bool BalancedSymbols(char open, char close) {
    if (open == '(' && close == ')') return true;
    if (open == '{' && close == '}') return true;
    if (open == '[' && close == ']') return true;
    return false;
}

// function to push charcaters inside the stack
void push(char c) {
    ParenStack[++top] = c;
}

// function to remove
char pop() {
    if(isEmpty()) return true;
    return ParenStack[top--];
}
```

Expression	Valid (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Enter expression: (A+B)+(C-D) The expression is balanced.	The expression is balanced because each opening and closing parenthesis have the same amount
((A+B)+(C-D)	N	Enter expression: ((A+B)+(C-D) The expression is not balanced.	The expression is not balanced because each opening parenthesis has an extra 1 with no closing parenthesis
((A+B)+[C-D])	Y	Enter expression: ((A+B)+[C-D]) The expression is balanced.	The expression is balanced because each opening and closing parenthesis and brackets have the same amount
((A+B]+[C-D]}	N	Enter expression: ((A+B]+[C-D]} The expression is not balanced.	The expression is not balanced because there are 2 extra closing

			brackets with no opening brackets

b. Linked Lists

Linked List

BalancedSymbols_LL.h > BalancedCheck<T>(T)

```h
#ifndef BALANCEDSYMBOLS_LL_H
#define BALANCEDSYMBOLS_LL_H
#include <iostream>
#include <string>

template <typename T>
class Node {
    public:
    T data;
    Node<T>* next = nullptr;
};

template <typename T>
Node<T>* top = nullptr;

template <typename T>
void push(T newData) {
    Node<T>* newNode = new Node<T>;
    newNode->data  = newData;
    newNode->next = top<T>;
    top<T> = newNode;
}

template <typename T>
bool isEmpty() {
    return top<T> == nullptr;
}

template <typename T>
T pop() {
    if (isEmpty<T>()) return true;
    Node<T>* temp = top<T>;
    T tempVal = temp->data;
    top<T> = top<T>->next;
    delete temp;
    return tempVal;
}

bool Balanced(char open, char closed) {
    if(open == '(' && closed == ')') return true;
    if(open == '{' && closed == '}') return true;
    if(open == '[' && closed == ']') return true;
    return false;
}

```

```h
template <typename T>
bool BalancedCheck(T newData) {
    bool balanced = true;
    for (int i = 0; newData[i] != '\0'; i++) {
        char ch = newData[i];
        if (ch == '(' || ch == '{' || ch == '[') {
            push<char>(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            char open = pop<char>();
            if (!Balanced(open, ch)) {
                balanced = false;
                break;
            }
        }
    }
    // output if expression is balanced or not
    if (balanced && isEmpty<char>())
        std::cout << "The expression is well balanced.";
    else
        std::cout << "The expression is not well balanced.";
    return balanced;
}
#endif
```

Main Program:

```
#include <iostream>
#include <string>
#include "BalancedSymbols_LL.h"

int main () {
    std::string expression;

    std::cout <<"Enter an expression: ";
    std::cin >> expression;
    BalancedCheck(expression);
}
```

Expression	Valid (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Enter an expression: (A+B)+(C-D) The expression is well balanced.	The expression is well balanced because every opening delimiter has a matching closing delimiters
((A+B)+(C-D)	N	Enter an expression: ((A+B)+(C-D) The expression is not well balanced.	The expression is not well balanced because one of the opening delimiters has no matching closing delimiter.
((A+B)+[C-D])	Y	Enter an expression: ((A+B)+[C-D]) The expression is well balanced.	The expression is well balanced because every opening delimiters has a matching closing delimiters, despite a different bracket being insde the expression
((A+B]+[C-D]}	N	Enter an expression: ((A+B]+[C-D]} The expression is not well balanced.	The expression is not well balanced because there the bracket are not set properly and a closing delimiter has no matching opening delimiter

Tools Analysis:

● How do the different internal representations affect the implementation and usage of the stack?

The code on how to efficiently use a stack differs when you're using it in an array, linked list, or using the stack STL, because there are different ways to construct the code and make it work. In arrays, you need a fixed amount of capacity, and you increment the top for push and decrement for pop. In linked lists, the capacity is already dynamic (capacity can change); you just need to create new memory for push and delete the memory for pop. When using the stack STL, despite not using it in this activity, it is the same as using it in arrays, but doesn't need a function for each method, since the method is already inside the STL.

Conclusion:

In conclusion, a stack is a data structure that can be manipulated using different methods. It follows LIFO (Last In, First Out) and can push, pop, peek/top, and determine if the stack is empty or full. Pop adds an item to the stack, push pushes it out of the stack, peek/top is used to see the top of the stack, and isEmpty/isFull determine if it is empty or full.

In this procedure, we learned how to use not only the stack STL, but also how to create similar methods without the STL module. We also learned how to implement similar methods for arrays and linked lists and make a program that lets you add and remove items inside the array and test out how the linked list stack works.

In this supplementary activity, stacks can also be used to determine if the expression is balanced by determining if the brackets have an opening and closing delimiter. Arrays were not easy to implement because in this activity, you have to implement another function that checks if the brackets have an opening and closing delimiter and checks if it satisfies it; the same thing with the linked lists.

Overall, I don't think I did well in the activity, and it took me a long time to do this. Without the help of AI and using forums, I have no way of debugging this, and my problems are different from what others are experiencing. I would really want to improve my understanding of stacks, especially implementing them in a linked list, because the linked list was the hardest part to make stacks, especially with additional functions to add to it.



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Using Grammarly to fix any mistakes or grammar issues in my tools analysis and conclusion**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.